

Advanced Optimization

Chapter 7 — Large-Scale Unconstrained Optimization

Hongseok Kim

Department of Electronic Engineering, Sogang University

Fall 2026

Reading: Nocedal & Wright, *Numerical Optimization*, 2nd ed., Chapter 7

What breaks when n is 10^5 – 10^6 , and what survives

Breaks

- ▶ Storing a dense $n \times n$ matrix (H_k of BFGS is dense even when $\nabla^2 f$ is sparse)
- ▶ $O(n^3)$ factorizations; fill-in during elimination
- ▶ Anything that touches all n^2 entries per iteration

Survives

- ▶ Nonlinear CG (§5.2) — unchanged; a few vectors
- ▶ Newton with *sparse* factorizations, when affordable and $\nabla^2 f$ formable
- ▶ Anything built from matrix–vector products

The chapter's map

- ▶ **7.1 Inexact Newton:** solve $\nabla^2 f_k p = -\nabla f_k$ *approximately* by CG — line search and trust region versions; Hessian-free.
- ▶ **7.2 Limited memory:** keep only m vector pairs instead of H_k — L-BFGS, two-loop recursion, compact representations.
- ▶ **7.3 Sparse quasi-Newton** (briefly); **7.4 partial separability**; **7.5 software**.

For a 4,500-bus AC power flow Jacobian or an OPF KKT system, this chapter is not “advanced topics” — it is the operating regime.

How inexact is allowed? The forcing sequence

Newton step: $\nabla^2 f_k p_k^N = -\nabla f_k$ (7.1). Solve it *iteratively* (CG), stopping early. Measure inexactness by the residual

$$r_k = \nabla^2 f_k p_k + \nabla f_k, \quad (7.2)$$

and terminate the inner iteration when

$$\|r_k\| \leq \eta_k \|\nabla f_k\|, \quad 0 < \eta_k < 1. \quad (7.3)$$

$\{\eta_k\}$ is the **forcing sequence**.

Two things bought immediately:

- ▶ No factorization, no fill-in — only products $\nabla^2 f_k d$.
- ▶ A dial: the tolerance controls how much work each outer iteration costs. The theory below says the same dial controls the *convergence rate*.

You choose the rate by choosing the tolerance (Theorems 7.1, 7.2)

Theorem 7.1

$\nabla^2 f$ continuous near x^* , $\nabla^2 f(x^*) \succ 0$, iteration $x_{k+1} = x_k + p_k$ with (7.3) and $\eta_k \leq \eta < 1$. For x_0 close enough to x^* : $x_k \rightarrow x^*$ linearly, with $\|\nabla^2 f(x^*)(x_{k+1} - x^*)\| \leq \hat{\eta} \|\nabla^2 f(x^*)(x_k - x^*)\|$, $\eta < \hat{\eta} < 1$.

Theorem 7.2

If moreover $x_k \rightarrow x^*$: $\eta_k \rightarrow 0 \Rightarrow$ **superlinear**; $\nabla^2 f$ Lipschitz and $\eta_k = O(\|\nabla f_k\|) \Rightarrow$ **quadratic**.

Where it comes from (the whole argument is three lines of Taylor):

$$\nabla f_{k+1} = \nabla f_k + \nabla^2 f_k p_k + o(\|p_k\|) = r_k + o(\|\nabla f_k\|) \quad \Rightarrow \quad \|\nabla f_{k+1}\| \leq (\eta_k + o(1)) \|\nabla f_k\|. \quad (7.5-7.6)$$

The residual you leave behind *is* next iteration's gradient, to first order. Everything follows from that sentence.

Practical choices

$\eta_k = \min(0.5, \sqrt{\|\nabla f_k\|})$: superlinear. $\eta_k = \min(0.5, \|\nabla f_k\|)$: quadratic.

Far from x^* , both keep $\eta_k = 0.5$ — crude solves where accuracy would be wasted; tight solves only for the endgame.

Algorithm 7.1 (Line Search Newton–CG)

```
for  $k = 0, 1, 2, \dots$ 
   $\epsilon_k = \min(0.5, \sqrt{\|\nabla f_k\|}) \|\nabla f_k\|$ ;  $z_0 = 0$ ,  $r_0 = \nabla f_k$ ,  $d_0 = -r_0$ ;
  for  $j = 0, 1, 2, \dots$ 
    if  $d_j^T B_k d_j \leq 0$ : return  $p_k = -\nabla f_k$  if  $j = 0$ , else  $p_k = z_j$ ; [negative curvature]
     $\alpha_j = r_j^T r_j / d_j^T B_k d_j$ ;  $z_{j+1} = z_j + \alpha_j d_j$ ;  $r_{j+1} = r_j + \alpha_j B_k d_j$ ;
    if  $\|r_{j+1}\| < \epsilon_k$ : return  $p_k = z_{j+1}$ ; [converged]
     $\beta_{j+1} = r_{j+1}^T r_{j+1} / r_j^T r_j$ ;  $d_{j+1} = -r_{j+1} + \beta_{j+1} d_j$ ;
   $x_{k+1} = x_k + \alpha_k p_k$ ,  $\alpha_k$  by Wolfe / Goldstein / backtracking, trying  $\alpha_k = 1$  first.
```

Differences from plain CG (Algorithm 5.2): start at $z_0 = 0$; stop at tolerance ϵ_k ; **stop on negative curvature**. The last exit makes p_k a descent direction always (at $j = 0$ it returns steepest descent). Preconditioning drops in as in Ch. 5.

The weakness

When $\nabla^2 f_k$ is *nearly* singular, the direction can be enormously long and poor — many function evaluations, little progress. Normalizing it or thresholding $d^T B d$ needs parameters nobody knows how to set. The trust-region version handles this cleanly — and is the one to prefer.

Hessian-free Newton

Algorithm 7.1 never needs $\nabla^2 f_k$ itself — only products $\nabla^2 f_k d$. Two sources:

► **Finite differences of the gradient:**

$$\nabla^2 f_k d \approx \frac{\nabla f(x_k + hd) - \nabla f(x_k)}{h}, \quad (7.10)$$

accuracy $O(h)$; cost **one extra gradient evaluation per CG iteration**. Choice of h : Ch. 8.

► **Automatic differentiation** (Ch. 8): exact Hessian–vector products at a small multiple of a gradient’s cost — forward-over-reverse mode. In ML frameworks this is a few lines (`hvp`, `jvp`, `vjp`).

So “Newton’s method needs second derivatives” is false at scale: it needs the Hessian’s *action*, which gradients alone can supply. The whole method touches $\nabla^2 f$ only through a black-box $d \mapsto \nabla^2 f d$.

Algorithm 7.2 (CG–Steihaug) — now with its explanation

Previewed in Ch. 4; the inner loop is Algorithm 5.2 with three exits. Subproblem:
 $\min_p m_k(p) = f_k + \nabla f_k^T p + \frac{1}{2} p^T B_k p$ s.t. $\|p\| \leq \Delta_k$, $B_k = \nabla^2 f_k$ (7.11).

Given $\epsilon_k > 0$; $z_0 = 0$, $r_0 = \nabla f_k$, $d_0 = -r_0$; if $\|r_0\| < \epsilon_k$ return $p_k = 0$;

for $j = 0, 1, 2, \dots$

if $d_j^T B_k d_j \leq 0$: find τ minimizing $m_k(z_j + \tau d_j)$ with $\|z_j + \tau d_j\| = \Delta_k$; return. [neg. curvature → boundary]

$\alpha_j = r_j^T r_j / d_j^T B_k d_j$; $z_{j+1} = z_j + \alpha_j d_j$;

if $\|z_{j+1}\| \geq \Delta_k$: find $\tau \geq 0$ with $\|z_j + \tau d_j\| = \Delta_k$; return. [hit the boundary]

$r_{j+1} = r_j + \alpha_j B_k d_j$; **if** $\|r_{j+1}\| < \epsilon_k$: return $p_k = z_{j+1}$; [converged]

$\beta_{j+1} = r_{j+1}^T r_{j+1} / r_j^T r_j$; $d_{j+1} = -r_{j+1} + \beta_{j+1} d_j$.

Outer loop: Algorithm 4.1 manages Δ_k and accepts/rejects. Near a good x^* the bound goes inactive and this is inexact Newton — Theorems 7.1/7.2 apply, same forcing sequence, fast endgame for free.

Why $z_0 = 0$ is the whole design

Fact 1: the first iterate is the Cauchy point

If $d_0^T B_k d_0 \leq 0$, the first exit returns $p = -\Delta_k \nabla f_k / \|\nabla f_k\|$ — the Cauchy point. Otherwise $z_1 = \alpha_0 d_0 = -\frac{\nabla f_k^T \nabla f_k}{\nabla f_k^T B_k \nabla f_k} \nabla f_k$ — the Cauchy point if interior, and if $\|z_1\| \geq \Delta_k$ the second exit returns the boundary Cauchy point. Later iterations only decrease m_k . Hence $m_k(p_k) \leq m_k(p_k^C)$: **global convergence via Theorem 4.4, automatically.**

Fact 2 (Theorem 7.3): the iterates only grow

$$0 = \|z_0\| < \cdots < \|z_j\| < \|z_{j+1}\| < \cdots < \|p_k\| \leq \Delta_k.$$

So the moment the boundary is reached, *no better interior point is coming* — stopping there is legitimate, not lazy.

Proof sketch — the payoff of Chapter 5: $z_j = \sum_{i < j} \alpha_i d_i$, and the expanding-subspace property (Thm 5.2) gives $z_j^T r_j = 0$; an induction using $\beta_{j+1} > 0$, $\alpha_j > 0$ gives $z_j^T d_j > 0$; then $\|z_{j+1}\|^2 = \|z_j\|^2 + 2\alpha_j z_j^T d_j + \alpha_j^2 \|d_j\|^2 > \|z_j\|^2$. When $B_k \succ 0$, the decrease achieved is at least *half* the optimal decrease of the subproblem.

Preconditioning Steihaug, and Algorithm 7.3

Precondition the inner CG as in Ch. 5; Theorem 7.3 then holds in the norm $\|D \cdot\|$, so the subproblem must use the *same* norm:

$$\min_p m_k(p) \quad \text{s.t. } \|Dp\| \leq \Delta_k \quad (7.13)$$

— the elliptical trust region of §4.5. Change variables $\hat{p} = Dp$ and run Algorithm 7.2 unmodified.

Algorithm 7.3 (Inexact Modified Cholesky) — the standard preconditioner

Scale $\bar{B} = T^{-1/2} B T^{-1/2}$ with $T = \text{diag}(\|B e_i\|)$; $\beta = \|\bar{B}\|$; $\alpha_0 = 0$ if $\min_i \bar{b}_{ii} > 0$, else $\beta/2$;

for $k = 0, 1, \dots$: attempt *incomplete* Cholesky $LL^T = \bar{B} + \alpha_k I$; success $\Rightarrow$ return L ; failure $\Rightarrow \alpha_{k+1} = \max(2\alpha_k, \beta/2)$.

Then $D = L^T$. This is Algorithm 3.3 and incomplete Cholesky fused into one loop — sparsity-preserving *and* indefiniteness-proof. Used in LANCELOT and TRON.

Beyond Steihaug: the Lanczos refinement (keyword)

Steihaug accepts *any* negative-curvature direction, even a nearly useless one (e.g. $m(p) = 10^{-3}p_1 - 10^{-4}p_1^2 - p_2^2$: it exits along p_1 for a reduction of $\sim 10^{-3}$ when 1 was available along p_2). The Newton–Lanczos method (GLTR) solves the subproblem more accurately in a growing Krylov basis — more per-iteration cost, better steps.

L-BFGS: the idea

BFGS keeps H_k — dense, n^2 storage, dead at large n . But look at what BFGS *does*:

$$H_{k+1} = V_k^T H_k V_k + \rho_k s_k s_k^T, \quad V_k = I - \rho_k y_k s_k^T, \quad \rho_k = \frac{1}{y_k^T s_k}. \quad (7.16-7.17)$$

H_k is just H_0 dressed with k correction pairs. So:

Limited memory

Store only the **m most recent pairs** $\{s_i, y_i\}$, $i = k - m, \dots, k - 1$ ($m \approx 3-20$), plus a fresh diagonal seed H_k^0 each iteration. Compute $H_k \nabla f_k$ *implicitly* from the pairs — never form any matrix. After the step, discard the oldest pair, store the newest.

Unrolling (7.16) m times gives the explicit expression (7.19):

$H_k = (V^T \dots) H_k^0 (\dots V) + \sum \rho_i (V^T \dots) s_i s_i^T (\dots V)$ — which collapses into a beautiful 8-line procedure.

Algorithm 7.4 (L-BFGS two-loop recursion)

Compute $r = H_k \nabla f_k$ from $\{s_i, y_i\}_{i=k-m}^{k-1}$ and H_k^0

$q \leftarrow \nabla f_k$;

for $i = k-1, k-2, \dots, k-m$: $\alpha_i \leftarrow \rho_i s_i^T q$; $q \leftarrow q - \alpha_i y_i$;

$r \leftarrow H_k^0 q$;

for $i = k-m, k-m+1, \dots, k-1$: $\beta \leftarrow \rho_i y_i^T r$; $r \leftarrow r + s_i(\alpha_i - \beta)$;

return $r = H_k \nabla f_k$.

- ▶ Cost: **4mn multiplications** (+n for diagonal H_k^0). Storage: $2m$ vectors. For $m = 10$, $n = 10^6$: forty million flops and eighty MB — laptop territory.
- ▶ The multiplication by H_k^0 is *isolated* between the loops — so the seed can change every iteration, or even be implicit (solve $B_k^0 r = q$).
- ▶ Backward loop peels updates off; forward loop puts them back. Deriving it from (7.19) once by hand, for $m = 2$, is the homework that makes it stick.

Algorithm 7.5 (L-BFGS), and what to expect from it

Choose x_0 , $m > 0$; $k \leftarrow 0$;

repeat: $H_k^0 = \gamma_k I$ with $\gamma_k = \frac{s_{k-1}^T y_{k-1}}{y_{k-1}^T y_{k-1}}$ (7.20); $p_k = -H_k \nabla f_k$ by Algorithm 7.4;

$x_{k+1} = x_k + \alpha_k p_k$ with a **Wolfe** line search (try $\alpha_k = 1$ first);

if $k > m$ discard $\{s_{k-m}, y_{k-m}\}$; store s_k, y_k ; $k \leftarrow k + 1$.

- ▶ γ_k is the scaling (6.20) applied *every* iteration — it sizes the seed like an eigenvalue of $\nabla^2 f^{-1}$ along the recent step, so $\alpha_k = 1$ is usually accepted.
- ▶ Wolfe conditions are still what keeps $y^T s > 0$ — nothing changed from Ch. 6.
- ▶ First $m - 1$ iterations $\equiv$ full BFGS (same H_0).
- ▶ Larger m : fewer iterations, more cost per iteration; diminishing returns past $m \sim 20$. Small m : cheap but less robust.

Weakness

Slow on *ill-conditioned* problems — wide eigenvalue spread is exactly what m pairs cannot capture. There, Hessian-free Newton-CG (with a preconditioner) wins; on cheap-function problems with moderate conditioning, L-BFGS wins; nonlinear CG is sometimes competitive.

L-BFGS and nonlinear CG are relatives

Write the Hestenes–Stiefel direction as a matrix acting on the gradient:

$$p_{k+1} = -\nabla f_{k+1} + \frac{\nabla f_{k+1}^T y_k}{y_k^T p_k} p_k = -\left(I - \frac{s_k y_k^T}{y_k^T s_k}\right) \nabla f_{k+1} \equiv -\hat{H}_{k+1} \nabla f_{k+1}. \quad (7.21)$$

$\hat{H}_{k+1}$: not symmetric, not positive definite, doesn't satisfy the secant equation. Fix all three in the minimal way and you get

$$H_{k+1} = \left(I - \frac{s_k y_k^T}{y_k^T s_k}\right) \left(I - \frac{y_k s_k^T}{y_k^T s_k}\right) + \frac{s_k s_k^T}{y_k^T s_k} \quad (7.22)$$

— **one BFGS update applied to I** : the **memoryless BFGS** method, i.e. L-BFGS with $m = 1$ and $H_k^0 = I$.

With exact line searches ($\nabla f_{k+1}^T p_k = 0$)

Memoryless BFGS = Hestenes–Stiefel = Polak–Ribière (7.23).

So the Ch. 5 methods sit at one end of a dial ($m = 0$ – 1) whose other end is full BFGS ($m = \infty$). L-BFGS is the dial. This is also why L-BFGS inherits CG's weakness on ill-conditioned problems: with small m , it nearly *is* CG.

Compact representation (Theorem 7.4) — the form the solvers use

k BFGS updates to B_0 , written at once as a low-rank correction:

$$B_k = B_0 - [B_0 S_k \quad Y_k] \begin{bmatrix} S_k^T B_0 S_k & L_k \\ L_k^T & -D_k \end{bmatrix}^{-1} \begin{bmatrix} S_k^T B_0 \\ Y_k^T \end{bmatrix}, \quad (7.24)$$

$S_k = [s_0 \cdots s_{k-1}]$, $Y_k = [y_0 \cdots y_{k-1}]$ ($n \times k$), $(L_k)_{ij} = s_{i-1}^T y_{j-1}$ for $i > j$ else 0, $D_k = \text{diag}(s_i^T y_i)$.

$$B_k = \delta_k I - \begin{array}{c} \boxed{n \times 2m} \\ \boxed{2m \times 2m} \\ \boxed{2m \times n} \end{array}$$

Limited-memory version (7.29): $B_0 \rightarrow \delta_k I$, windows S_k, Y_k of the last m pairs. Update: $2mn$ ops; product $B_k v$: $(4m+1)n$; the $2m \times 2m$ middle factorization is negligible.

Why this matters beyond Ch. 7

Trust regions and *constrained* solvers need B_k (not H_k) and need projections of it. **L-BFGS-B, KNITRO, and IPOPT all carry their quasi-Newton Hessians in exactly this form** (see §19.3). SR1 has an analogous compact form (7.30), self-dual under $B \leftrightarrow H$, $s \leftrightarrow y$; limited-memory SR1 is sometimes weaker — no positive definiteness near x^* .

Sparse quasi-Newton, and why it disappointed

Intuitively appealing: demand that B_{k+1} have the *sparsity pattern of the true Hessian*

$$\Omega = \{(i, j) : [\nabla^2 f(x)]_{ij} \neq 0 \text{ somewhere}\},$$

and among such matrices satisfy the secant equation, closest to B_k in Frobenius norm. The solution requires solving an $n \times n$ sparse linear system per update — feasible.

Practice

- ▶ The update is *not scale invariant* under diagonal scalings, and behaves badly.
- ▶ B_{k+1} may be indefinite or ill-conditioned even when the secant pair is good; curvature gets forced into a pattern that cannot hold it.
- ▶ Numerical results: disappointing. The idea survives mainly as a cautionary tale — *imposing structure on an approximation is not the same as having structure in the object.*

The structural idea that *does* work is the next one.

Partially separable functions: exploit the sum, not the pattern

Most large objectives are sums of **element functions**, each touching few variables:

$$f(x) = \sum_{i=1}^{ne} \phi_i(U_i x), \quad (7.40)$$

U_i a compactifying matrix selecting the element variables (e.g. rows of I). Then $\nabla f = \sum U_i^T \nabla \phi_i$,
 $\nabla^2 f = \sum U_i^T \nabla^2 \phi_i U_i$ (7.37).

Element-wise quasi-Newton

Maintain a *tiny dense* approximation $B_{[i]} \approx \nabla^2 \phi_i$ per element (from element pairs $s_{[i]}, y_{[i]}$, by BFGS or SR1), and assemble

$$B = \sum_{i=1}^{ne} U_i^T B_{[i]} U_i \quad (7.41)$$

— never explicitly: Bv products go element by element, inside trust-region CG.

- ▶ $n = 1000$, elements of dimension 2: each block is *fully determined after a few steps* — the assembled B becomes excellent fast, while a structure-blind method must estimate a 1000×1000 matrix from the same information.
- ▶ SR1 is natural here — $y_{[i]}^T s_{[i]} > 0$ cannot be enforced elementwise (the promised use case from Ch. 6).
- ▶ The structure must be *known*: modeling language, or the code's own decomposition. A grid objective (a sum over branches and buses) has it for free.

Perspectives and software

Method	Derivatives	Storage	Strength	Codes
Newton, sparse factor.	$\nabla^2 f$ explicit	fill-dependent	fast, robust	(sparse LDL ^T)
Newton–CG (7.1/7.2)	$\nabla^2 f d$ products	few vectors	scales; indefinite OK	TRON, LANCELOT
L-BFGS (7.5)	∇f only	$2m$ vectors	cheap, simple, robust	L-BFGS, L-BFGS-B
Nonlinear CG (§5.2)	∇f only	few vectors	minimal memory	CG+
Partially separable QN	∇f + structure	element blocks	fast, robust	LANCELOT

Rules of thumb

Hessian formable and factorable $\Rightarrow$ just use Newton with sparse factorization. Hessian products only $\Rightarrow$ Newton–CG (trust region), preconditioned if possible. Gradients only, moderate conditioning $\Rightarrow$ L-BFGS first — it is the default for a reason. Structure known $\Rightarrow$ exploit it; nothing beats it when available.

And the compact representations of §7.2 are how large-scale *constrained* solvers (IPOPT, KNITRO with `hessian_approximation=limited-memory`) carry curvature — the option you toggle when the true Hessian of an OPF Lagrangian is too expensive.

Chapter 7 in one table

Result / object	Says
Forcing sequence (7.3)	inner-solve tolerance is a rate dial
Theorems 7.1/7.2	η bounded: linear; $\eta_k \rightarrow 0$: superlinear; $O(\ \nabla f\)$: quadratic
Algorithm 7.1	Newton-CG line search; neg. curvature $\rightarrow$ steepest descent
Algorithm 7.2 (Steihaug)	three exits; $z_0 = 0 \Rightarrow$ Cauchy decrease guaranteed
Theorem 7.3	$\ z_j\ $ strictly increasing $\Rightarrow$ stopping at the boundary is optimal
Algorithm 7.3	incomplete modified Cholesky preconditioner (LANCELOT, TRON)
Two-loop recursion (7.4)	$H_k \nabla f_k$ in $4mn$ flops from $2m$ vectors
L-BFGS (7.5)	γ_k seed + Wolfe; first $m-1$ iters = BFGS
Memoryless BFGS (7.22)	$m = 1$, $H^0 = I$; exact LS $\Rightarrow$ HS = PR
Compact form (7.24/7.29)	outer-product B_k ; used by L-BFGS-B, KNITRO, IPOPT
Partial separability (7.40)	element-wise QN: small dense blocks, assembled products

Two ideas to carry forward

The residual you tolerate in the inner solve is next iteration's gradient — inexactness is a rate choice, not a compromise. A quasi-Newton matrix is its history of pairs; store the history, not the matrix.

Homework

Derivations

- 1 Fill in the derivation (7.5)–(7.7) of Theorem 7.1, marking where continuity of $\nabla^2 f$ and $\eta_k < 1$ are used; then derive the quadratic case of Theorem 7.2.
- 2 Verify that the first iterate of Algorithm 7.2 is the Cauchy point (all three cases).
- 3 Complete the induction $z_j^T d_j > 0$ in Theorem 7.3.
- 4 Show that with exact line searches, memoryless BFGS (7.22) reduces to Hestenes–Stiefel and Polak–Ribière (7.23).
- 5 Unroll Algorithm 7.4 by hand for $m = 2$ and verify it computes (7.19) applied to ∇f_k .
- 6 Check Theorem 7.4 for $k = 1$: the compact form (7.24) equals one BFGS update (6.19) of B_0 .

Implementation

7. **The 40-line one:** implement the two-loop recursion and L-BFGS (Alg. 7.5) with your Ch. 3 Wolfe search. Extended Rosenbrock, $n = 10^4$: run $m \in \{1, 3, 5, 10, 20\}$, plot f vs. gradient evaluations. Compare with your Ch. 5 PR+ (that is the $m=1$ end of the dial).
8. Hessian-free trust-region Newton-CG: Algorithm 4.1 + Algorithm 7.2 with $\nabla^2 f d$ by finite differences (7.10). Same problem; compare inner-iteration counts with and without a diagonal preconditioner.
9. Verify the rate dial: run item 8 with $\eta_k = 0.5$ fixed vs. $\eta_k = \min(0.5, \sqrt{\|\nabla f_k\|})$; $\log \|\nabla f_{k+1}\| / \|\nabla f_k\|$ near the solution.

Next: Ch. 8 — where derivatives come from: finite differences, AD, and the adjoint.